

第八单元: Typescript

1. 什么是TypeScript?
2. TypeScript 的优势
3. 安装和编译ts
4. TS基础数据类型
5. 定义函数类型
 - 5.1. 定义函数类型
 - 5.2. 可选参数
 - 5.3. 默认参数
 - 5.4. 剩余参数
 - 5.5. 函数重载
6. 定义对象类型
 - 6.1. 对象字面量类型
 - 6.2. 类型别名定义对象类型
 - 6.3. interface定义对象类型
7. interface 接口
 - 7.1. 定义对象类型 (详情查看6.3)
 - 7.2. 定义函数类型
 - 7.3. 定义数组类型
 - 7.4. 继承
8. type 和 interface
 - 8.1. 相同点
 - 8.2. 不同点
9. 联合类型和交叉类型
 - 9.1. 联合类型 (Union Types)
 - 9.2. 交叉类型 (Intersection Types)
10. 类型断言和类型推论
 - 10.1. 类型推论 (Type Inference)
 - 10.2. 类型断言 (Type Assertion)

- 11. 枚举
- 12. 泛型
 - 12.1. 定义泛型函数
 - 12.2. 泛型别名
 - 12.3. 泛型接口
 - 12.4. 泛型约束
- 13. keyof和typeof
 - 13.1. typeof
 - 13.2. keyof
 - 13.3. 组合使用
- 14. 内置泛型
 - 14.1. Partial<T>
 - 14.2. Required<T>
 - 14.3. Readonly<T>
 - 14.4. Pick<T, K>
 - 14.5. Omit<T, K>
 - 14.6. Exclude<T, U>
 - 14.7. Extract<T, U>
 - 14.8. ReturnType<T>
- 15. 类型声明文件
- 16. 定义全局变量类型

1. 什么是TypeScript?

- TypeScript是一个由Microsoft开发的开源编程语言，它在JavaScript的基础上添加了静态类型系统。作为JavaScript的一个超集，TypeScript包含了JS的所有功能，同时引入了类型注解和编译时类型检查。
- TS无法直接在浏览器上运行，需编译成JS来运行。

2. TypeScript 的优势

1. 与JavaScript的兼容性

- TypeScript完全兼容JavaScript的代码和库，这意味着任何现有的JavaScript代码可以无缝地在TypeScript项目中使用，反之亦然。
- 这种兼容性确保了开发者可以渐进式地采用TypeScript，而不需要对现有项目进行大规模重写。

2. 静态类型检查

- 静态类型检查可以在代码运行之前发现潜在的类型错误，减少运行时错误的发生。
- 类型注解和接口提供了一种方式来定义变量、函数参数和返回值的类型，使得代码更加清晰和易于理解。

3. 增强的代码质量和可维护性

- 类型系统和编译时检查强化了代码质量，使得开发过程中的错误更容易被发现和修正。
- 类型注解也充当了代码的文档，提高了代码的可读性和维护性。

4. IDE支持和开发体验

- 大多数现代IDE和编辑器都对TypeScript提供了优秀的支持，包括自动完成、代码导航、重构工具和实时的错误检测。
- 这些特性显著提升了开发效率和体验。

3. 安装和编译ts

1. 全局安装编译工具 `npm i -g typescript`
2. 查看工具的版本号 `tsc -v`
3. 生成 tsconfig.json, 编译时的规则配置 `tsc --init`
4. 监听 ts 文件改变，实时编译成 js `tsc --watch`

4. TS基础数据类型

typescript 引入了静态类型检查。这意味着你可以在编写代码时指定变量、参数、返回值等的类型。这样不仅能提前发现潜在的错误，还能提高代码的可读性和可维护性。

TS的基础数据类型：

1. **boolean**: 最基本的数据类型，只有两个值：**true**和**false**。

```
1 let isDone: boolean = false;
```

2. **number**: 所有数字，包括整数和浮点数，都是**number**类型，支持十进制、十六进制、二进制和八进制字面量。

```
1 let decimal: number = 6;
2 let hex: number = 0xf00d;
3 let binary: number = 0b1010;
4 let octal: number = 0o744;
```

3. **string**: 用于表示文本数据，可以使用双引号 (" ")、单引号 (' ') 或者模板字符串 (` `)。

```
1 let color: string = "blue";
2 color = 'red';
3 let fullName: string = `Bob Bobbington`;
4 let sentence: string = `Hello, my name is ${ fullName }.`;
```

4. **array**: TypeScript中数组可以有两种方式定义：一种是使用元素类型后面接上[], 另一种是使用数组泛型Array<元素类型>。

```
1 let list: number[] = [1, 2, 3];
2 let listGeneric: Array<number> = [1, 2, 3];
```

5. **tuple**: 元组类型允许表示一个已知元素数量和类型的数组，各元素的类型不必相同。

```
1 let x: [string, number];
2 x = ["hello", 10]; // OK
```

6. **enum**: 枚举类型是对JavaScript标准数据类型的一个补充，让我们可以为的一组数值赋予友好的名字。

```
1 enum Color {Red, Green, Blue}
2 let c: Color = Color.Green;
```

7. **any**: 对于那些我们不想给出一个具体类型的变量，可以使用**any**类型，相当于放弃类型检查。

```
1 let notSure: any = 4;
2 notSure = "maybe a string instead";
3 notSure = false; // okay, definitely a boolean
```

8. **void**: 与**any**相反，**void**用于那些没有任何类型的情况，通常用在没有返回值的函数。

```
1 function warnUser(): void {
2     console.log("This is my warning message");
3 }
```

9. **null** 和 **undefined**: TypeScript里，**undefined**和**null**两者各自有自己的类型分别叫做**undefined**和**null**。

```
1 let u: undefined = undefined;
2 let n: null = null;
```

10. **never**: 表示的是那些永不存在的值的类型，例如，**never**类型是那些总是会抛出异常或根本就不会有返回值的函数表达式或箭头函数表达式的返回值类型。

```
1 function error(message: string): never {
2     throw new Error(message);
3 }
```

11. **unknown**: 类型代表任何值，与**any**类型相似，但是**unknown**类型更加安全使用属性和方法前必须使用类型断言或者类型守卫确定类型。

```
1  let value: unknown = "hello";
2
3  // 直接使用value会出错, 因为它的类型是unknown
4  // console.log(value.length); // Error: Object is of type 'unknown'.
5
6  // 使用类型断言
7  if (typeof value === "string") {
8      console.log(value.length); // 正确: value被识别为string
9  }
10
11 // 或者使用类型守卫
12 if (value instanceof Date) {
13     // 在这个代码块中, value的类型被识别为Date
14     console.log(value.toISOString());
15 } else {
16     // 在这里, value的类型仍然是unknown
17 }
```

5. 定义函数类型

5.1. 定义函数类型

```
1 // 函数声明式
2 function fn1(a: number, b: string): number {
3     return a + b.length
4 }
5 // 函数表达式
6 const fn2 = function(a: number, b: string): number {
7     return a + b.length
8 }
9 const fn3 = (a: number, b: string): number => {
10     return a + b.length
11 }
12 // 对象中的函数
13 const obj = {
14     fn4(a: number, b: string): number {
15         return a + b.length
16     }
17 }
18 // 函数表达式的完整写法
19 const fn5: (a: number, b: string) => number = (a: number, b: string): number => {
20     return a + b.length
21 }
```

5.2. 可选参数

可选参数必须跟在必须参数后面。如果调用时没有提供可选参数，它的值将是`undefined`。

```
1 function buildName(firstName: string, lastName?: string) {
2     if (lastName) {
3         return firstName + " " + lastName;
4     } else {
5         return firstName;
6     }
7 }
8
9 let result1 = buildName("Bob"); // works correctly now
10 let result2 = buildName("Bob", "Adams"); // ah, just right
```

5.3. 默认参数

当调用函数而没有传递这个参数或者传递的参数值是`undefined`时，会使用该默认值。

```
TypeScript |
1 function buildName(firstName: string, lastName = "Smith") {
2     return firstName + " " + lastName;
3 }
4
5 let result1 = buildName("Bob");           // returns "Bob Smith"
6 let result2 = buildName("Bob", undefined); // still returns "Bob Smith"
7 let result3 = buildName("Bob", "Adams");   // returns "Bob Adams"
```

5.4. 剩余参数

```
TypeScript |
1 function buildName(firstName: string, ...restOfName: string[]) {
2     return firstName + " " + restOfName.join(" ");
3 }
4
5 let employeeName = buildName("Joseph", "Samuel", "Lucas", "MacKinzie");
```

5.5. 函数重载

为同一个函数提供多个函数类型定义，根据调用函数时使用的参数类型来确定使用哪个重载

```
TypeScript |
1 // 函数重载声明
2 function fn7(a: string): number
3 function fn7(a: number, b: number): string
4 // 函数实现
5 function fn7(a: any, b?: any): any {
6     if (typeof a === 'string') {
7         return a.length
8     } else {
9         return a.toFixed(b)
10    }
11 }
12 // 函数调用示例
13 const a = fn7('abcdefg')
14 const b = fn7(100, 2)
```


6. 定义对象类型

6.1. 对象字面量类型

直接在变量声明时指定对象的类型。这种方法适用于一次性的对象结构，不需要复用。

```
1  let car: {  
2    readonly make: string;  
3    model: string;  
4    year?: number; // 可选属性  
5    say: (hobby: string) => void;  
6  } = {  
7    make: "Toyota",  
8    model: "Corolla",  
9    year: 2020,  
10   say(hobby: string){}  
11  };
```

6.2. 类型别名定义对象类型

类型别名允许你给一个类型起个新名字，方便该类型重复使用。

```
1  type Point = {  
2    x: number;  
3    y: number;  
4  };  
5  const point1: Point = {  
6    x: 10,  
7    y: 20  
8  };  
9  const point2: Point = {  
10   x: 20,  
11   y: 10  
12  };
```

6.3. interface定义对象类型

```
1 interface User {  
2     id: number;  
3     name: string;  
4     email?: string; // 可选属性  
5 }  
6  
7 const user: User = {  
8     id: 1,  
9     name: "John Doe",  
10    email: "john.doe@example.com"  
11 };
```

7. interface 接口

7.1. 定义对象类型 ([详情查看6.3](#))

7.2. 定义函数类型

```
1 interface SearchFunc {  
2     (source: string, subString: string): boolean;  
3 }  
4  
5 let mySearch: SearchFunc = function(source: string, subString: string) {  
6     return source.search(subString) > -1;  
7 }
```

7.3. 定义数组类型

```
1 interface StringArray {  
2     [index: number]: string;  
3 }  
4  
5 let myArray: StringArray = ["Bob", "Fred"];
```

7.4. 继承

接口可以通过继承一个或多个其他接口来增加新的属性和方法。这是通过`extends`关键字实现的，支持多接口继承。

```
1  interface Shape {
2      color: string;
3  }
4  interface PenStroke {
5      penWidth: number;
6  }
7
8  // 继承单接口
9  interface Square extends Shape {
10     sideLength: number;
11 }
12 let square: Square = { color: "blue", sideLength: 10 };
13
14 // 继承多接口
15 interface Square extends Shape, PenStroke {
16     sideLength: number;
17 }
18 let square: Square = { color: "red", sideLength: 20, penWidth: 5.0 };
```

8. type 和 interface

在TypeScript中，`type`和`interface`都可以用于定义类型，但它们在某些方面有着不同的使用场景和特性。`interface`是定义类型，`type`是给现有的类型定义别名。

8.1. 相同点

- 都可以描述一个对象或函数的形状。
- 都可以被扩展。

8.2. 不同点

- `type` 可以声明基本类型、联合类型，而`interface`不行。
- `interface` 通过继承扩展，`type`通过交叉类型组合多个类型的方式扩展。

- interface重复定义时会自动合并，而 type 重名会报错。

```
1  interface User {
2      name: string;
3  }
4
5  interface User {
6      age: number;
7  }
8
9  // User 现在拥有 name 和 age 属性
10 const obj: User = {
11     name: '',
12     age: 20
13 }
```

9. 联合类型和交叉类型

9.1. 联合类型 (Union Types)

联合类型表示一个值可以是几种类型之一。它通过使用|符号连接类型来定义。

```
1  type mixedType = string | number
2  let a: mixedType = ''
3  let b: mixedType = 100
4
5  function process(input: mixedType) {
6      // 使用类型守卫缩小联合类型的范围
7      if (typeof input === "string") {
8          console.log(input.toUpperCase());
9      } else {
10         console.log(input.toFixed(2));
11     }
12 }
```

注意事项：

1. 公共属性：只能安全访问所有成员类型共有的属性或方法。对于特定类型的属性，需要先缩小类型范围。

2. 类型守卫：在处理联合类型的值时，通常需要使用类型守卫来确保类型安全的访问或操作。

9.2. 交叉类型 (Intersection Types)

交叉类型表示一个值同时拥有多种类型。它通过使用&符号来组合类型，允许你将多个类型合并成一个类型。

```
1  interface Runnable {
2      run(): void;
3  }
4
5  interface Flyable {
6      fly(): void;
7  }
8
9  type ActionHero = Runnable & Flyable;
10
11 function actionHero(hero: ActionHero) {
12     hero.run();
13     hero.fly();
14 }
```

注意事项：

1. 属性冲突：如果不同类型中包含相同名称但类型不兼容的属性，则会导致类型错误。在设计交叉类型时，需要确保所有类型可以兼容地合并。

10. 类型断言和类型推论

10.1. 类型推论 (Type Inference)

类型推论是TypeScript的一个特性，它允许编译器自动地推断出表达式的类型，从而无需显式指定。TypeScript会根据变量的使用上下文来推断其类型，这使得代码更简洁并减少了编码工作量。

- 初始化时推断：当你初始化变量时，TypeScript会根据赋值的数据类型推断变量的类型。

```
1  let num = 5; // 类型推断为 number
```

- **最佳通用类型**：当需要从多个表达式推断类型时，TypeScript会计算一个“最佳通用类型”来兼顾所有候选类型。

```
▼ TypeScript |
1 let arr = [1, 2, null]; // 类型推断为 (number | null)[]
```

- **上下文类型**：在表达式的类型由其所处的位置决定时，TypeScript会使用上下文来推断类型。

```
▼ TypeScript |
1 window.onmousedown = function(event) {
2     // event 被推断为 MouseEvent 类型
3 };
```

10.2. 类型断言 (Type Assertion)

类型断言是一种方式，允许你告诉编译器：“我知道这个变量的类型是什么。”类型断言像是一种方式，用于“重载”变量或对象的类型。当你比TypeScript更了解某个值的类型时，这会非常有用。

使用场景

- 当你确定变量的类型比TypeScript推断的类型更具体时。
- 当TypeScript无法确切知道一个联合类型的具体类型，但你确定知道时。

TypeScript中类型断言有两种形式：

```
▼ TypeScript |
1 // 尖括号语法
2 let someValue: any = "this is a string";
3 let strLength: number = (<string>someValue).length;
4
5 // as语法
6 let someValue: any = "this is a string";
7 let strLength: number = (someValue as string).length;
```

注意：在JSX中，只能使用 **as** 语法进行类型断言，因为尖括号语法与JSX的标签语法冲突。

11. 枚举

在TypeScript中，枚举（Enums）是一种特殊的类型，它允许为一组数值赋予友好的名字。枚举的主要作用是提升代码的可读性和可维护性，使数值集合更易于理解和使用。

作用：

1. **可读性**：使用枚举可以使代码更易于阅读和理解。使用单词给数值命名，代码的意图变得更加明显。
2. **易于维护**：枚举提供了一个集中的位置定义一组相关的常量。如果以后需要更改这些值，只需在枚举定义中修改即可。
3. **减少错误**：使用枚举可以避免魔术字符串（magic strings）或数字，减少因拼写错误或值错误引入的bug。

如何使用：

1. **数字枚举**：数字枚举的值默认从0开始自动增加，但你也可以手动指定成员的数值。

```
TypeScript |  
1 enum Direction {  
2     Up = 1,  
3     Down,  
4     Left,  
5     Right  
6 }  
7 console.log(Direction.Up);    // 输出: 1  
8 console.log(Direction.Left);  // 输出: 3
```

字符串枚举：要求每个成员都必须用字符串字面量或另一个字符串枚举成员初始化。

```
TypeScript |  
1 enum Direction {  
2     Up = "UP",  
3     Down = "DOWN",  
4     Left = "LEFT",  
5     Right = "RIGHT"  
6 }  
7 console.log(Direction.Up);    // 输出: "UP"
```

12. 泛型

泛型允许你在定义函数、接口或类时不具体指定数据类型，而是在使用时指定类型。可以理解为定义类型时的形参。

12.1. 定义泛型函数

```
1 // val 可以传入任意类型，次函数返回指定类型和长度的数组
2 function createArray<T>(val: T, num: number): T[] {
3     return new Array(num).fill(val)
4 }
5 let a1 = createArray<string>('abc', 10)
6 let b1 = createArray<number>(100, 10)
7 let c1 = createArray<{ name: string }>({ name: '小明' }, 10)
```

12.2. 泛型别名

```
1 // 定义多维数组类型
2 type deepArr<T> = T | deepArr<T>[]
3 const arr1: deepArr<number>[] = [[1, [2, [3, 4, 5], 6], 7, 8], 9]
4 const arr2: deepArr<string>[] = [['a', ['b'], ['c', 'd'], 'e'], 'f']
5
```

12.3. 泛型接口


```

1 // hobby属性的类型不确定，调用时再传入指定的类型
2 interface Person<T> {
3     name: string;
4     age: number;
5     hobby: T
6 }
7 const obj: Person<string> = {
8     name: '小明',
9     age: 33,
10    hobby: '吃饭'
11 }
12 const obj1: Person<number> = {
13     name: '小明',
14     age: 33,
15     hobby: 100
16 }

```

12.4. 泛型约束

限制泛型传入的类型可以通过泛型约束实现，使用`extends`关键字。

```

1 // 泛型约束：泛型 T 必须符合 extends 后的条件
2 // 限制 T 必须符合 { length: number } 类型，必须有 length 属性且值是数字
3 function getLen<T extends { length: number }>(a: T): number {
4     return a.length
5 }
6 console.log(getLen<string>('100'))
7 console.log(getLen<number[]>([1,2,3,4,5,6,7]))
8 console.log(getLen<{ a: number, length: number }>({ a: 100, length: 10 }))

```

在这个例子中，`getLen`要求其类型参数`T`必须符合`Lengthwise`接口，即必须包含`length`属性。

13. keyof和typeof

13.1. typeof

typeof操作符用于获取一个变量或对象的类型。在JavaScript中，**typeof**可以告诉你一个值的基础类型（如"number"、"string"等）。而在TypeScript中，**typeof**还被用于捕获变量的类型，以便在类型注解中使用。

```
1 let exampleVar = "hello world";
2 // 使用 typeof 获取 exampleVar 的类型
3 // exampleVarType 的类型为 string
4 type exampleVarType = typeof exampleVar
5
6 // 获取对象的类型
7 const obj = { name: '小明', age: 22 }
8 type IObj = typeof obj // { name: string; age: number; }
```

13.2. keyof

keyof 操作符用于取得一个对象类型的所有 **key** 的联合类型。简单地说，它会返回由该对象所有 **key** 组成的联合类型。

```
1 interface Person {
2     name: string;
3     age: number;
4 }
5
6 // 使用 keyof 获取 Person 接口的所有键作为联合类型
7 type PersonKeys = keyof Person; // "name" | "age"
```

13.3. 组合使用

keyof和**typeof**可以组合使用，以创建更复杂的类型表达式。

在这个例子中，先使用**typeof**捕获**person**对象的类型，然后通过**keyof**获取这个类型的所有键，结果是一个包含"age"和"name"的联合类型。

```
1 const person = {  
2   name: "John",  
3   age: 30  
4 };  
5  
6 // 使用 typeof 获取 person 的类型, 然后使用 keyof 获取该类型的键的联合  
7 type PersonKeys = keyof typeof person; // "name" | "age"  
8 // 现在 PersonKeys 类型是 "name" | "age"
```

14. 内置泛型

14.1. Partial<T>

作用：把参数T类型的所有属性改成可选属性。

```
1 interface User {  
2   id: number;  
3   name: string;  
4   email: string;  
5 }  
6  
7 let partialUser: Partial<User> = {  
8   email: "example@example.com"  
9 };
```

14.2. Required<T>

作用：与Partial相反，将参数T类型的所有属性设置为必填属性。

```
1 interface Props {  
2   id?: number;  
3   name?: string;  
4 }  
5  
6 let props: Required<Props> = {  
7   id: 1,  
8   name: "Required Name"  
9 };
```

14.3. Readonly<T>

作用：将参数T类型的所有属性设置为只读属性。

14.4. Pick<T, K>

作用：从类型T中选择一组属性来构造类型，获取对象类型中的指定属性。

```
1 interface Todo {  
2   id: number;  
3   title: string;  
4   completed: boolean;  
5 }  
6  
7 let todoItem: Pick<Todo, "id" | "title"> = {  
8   id: 1,  
9   title: "Buy milk"  
10 };
```

14.5. Omit<T, K>

作用：剔除对象类型中的指定属性。

```
1 interface Todo {
2   id: number;
3   title: string;
4   completed: boolean;
5 }
6
7 let todoItem: Omit<Todo, "completed"> = {
8   id: 1,
9   title: "Buy milk"
10 };
```

14.6. Exclude<T, U>

作用：从类型T中排除可以赋值给类型U的类型。

```
1 type T = string | number | boolean;
2 type U = string | number;
3
4 type Result = Exclude<T, U>; // boolean
```

14.7. Extract<T, U>

作用：从类型T中提取出可以赋值给类型U的所有类型，提取T和U中的公共类型。

```
1 type T = string | number | boolean;
2 type U = string | boolean;
3 type CommonTypes = Extract<T, U>; // string | boolean
```

14.8. ReturnType<T>

作用：用于获取一个函数类型的返回类型。

```
1 function add(a: number, b: number): number {  
2     return a + b;  
3 }  
4 type ResultType = ReturnType<typeof add>;  
5 // ResultType 的类型是 number
```

15. 类型声明文件

1. typescript中有两种文件类型

- a. .ts 文件：可以定义类型和逻辑
- b. .d.ts 类型声明文件：只能定义类型

2. typescript 默认会直接读取全局的 .d.ts 文件中的类型

- a. .d.ts 文件中没有使用 import 或者 export 就是全局文件

3. 在 ts 文件中引入第三方包

- a. 自带声明文件，例如：axios
- b. 没有声明文件，例如：mockjs, 尝试从 npm 下载对应的声明文件 `npm i --save-dev @types/包名`
- c. 如果 ``npm i --save-dev @types/包名`` 没有对应的声明文件，需要手动定义对应的模块类型

16. 定义全局变量类型

```
1 // 在全局的 .d.ts 文件中
2 declare let myBlockScopedVar: number;
3 declare const myConstantVar: boolean;
4 declare function testfn(a: string): number;
5 // 扩展Window对象
6 declare global {
7     interface Window {
8         myGlobalFunction: () => void;
9     }
10 }
11
12 // 现在你可以像使用其他window属性一样使用myGlobalFunction
13 window.myGlobalFunction = () => console.log("This is a global function");
```